

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

Hyderabad Campus

Department of Computer Science and Information Systems

Mid-Semester Project Report

Software Localization Pipeline for Multilingual Resource Translation

Token-Efficient Batching and a Persistent Translation Memory on AWS

Submitted as part of the requirements for

Research Practice

Under the supervision of

Prof. Subhrakanta Panda

Submitted by

Name	ID
Tanish Kumar Gupta	2025H1030061H
Aayush Gupt	2025H1030042H
Alok Singh	2025H1030067H
Akshad Ajay Kalaskar	2025H1030065H

Mid-Semester Submission · Academic Year 2026–2027

Certificate

This is to certify that the project titled **Software Localization Pipeline for Multilingual Resource Translation** has been carried out by **Tanish Kumar Gupta**, **Aayush Gupt**, **Alok Singh** and **Akshad Ajay Kalaskar** under my supervision as part of the Research Practice requirement at BITS Pilani, Hyderabad Campus.

The work presented in this report continues the previous semester's localization pipeline. It moves the pipeline to Amazon Web Services, profiles its translation stage, redesigns its batching logic for token efficiency, adds safeguards for software placeholders and untranslatable characters, and rebuilds its Translation Memory so that continuous-integration runs translate only what has changed. Every result is backed by recorded runs on the KDE4 software-localization corpus in twelve languages.

Prof. Subhrakanta Panda
Department of Computer Science and
Information Systems
BITS Pilani, Hyderabad Campus

Acknowledgement

We thank **Prof. Subhrakanta Panda** for his guidance throughout the semester. His insistence on measuring every change, and on separating what was measured from what was estimated, shaped the experimental method used in this report.

We thank the **Department of Computer Science and Information Systems**, BITS Pilani, Hyderabad Campus, for the academic environment and resources that made this work possible, and the Track 1 team for preparing and cleaning the twelve-language KDE4 dataset used throughout our evaluation.

We also acknowledge the open-source communities behind **NLLB-200**, **CTranslate2**, **Hugging Face Transformers**, **SacreBLEU**, **COMET**, the **OPUS KDE4** corpus and the **FLORES-200** benchmark.

Abstract

This mid-semester report documents the progress made so far in the second semester of work on an automated software-localization pipeline that translates `.resx` resource files into multiple languages inside a continuous-integration (CI/CD) workflow. The pipeline was moved from Microsoft Azure to Amazon Web Services (AWS) with its translation logic unchanged, and then optimized in three steps, each measured on the KDE4 software-localization corpus in twelve languages.

Batching. Profiling showed that model inference takes 99.3% of the translation stage, that mixed-length batches wasted 30% of the model's input on padding, and that the machine's two cores were only 59% busy. Sorting strings by length cut padding to 4% but gained only 3%: padding was not the bottleneck. Reducing the beam size from 5 to 2 made translation 2.2× faster for a 0.45-point chrF loss, and translating two batches in parallel with an output length cap added 1.27× with identical output. A full production run translated all 10,054 usable KDE4 strings into 12 languages (120,648 translations) in 3.6 hours with no failures, about 2.5× faster than the semester's starting configuration, at chrF 63.29, spBLEU 47.90 and COMET 0.862.

Quality safeguards. Placeholder protection raised the share of strings whose placeholders survive translation from 87.7% to 99.88%. The run also revealed that 6.1% of Chinese outputs silently lost characters the model cannot write; these strings are now kept in English and reported instead of being shipped broken.

Translation Memory. The memory was rebuilt to persist between runs, to record which model and settings produced each translation, to load once per language per run, and to reuse near-identical strings. In a simulated CI/CD history, commits that change 1%, 5% and 10% of an application ran 25×, 14× and 8.4× faster than the previous pipeline with 98.4%, 95.0% and 90.6% fewer model tokens, at unchanged quality (chrF 63.38 versus 63.37, spBLEU 48.07 versus 48.04). Removing the remaining per-run set-up costs then cut a 5% commit, measured end to end, from 5.9 to 3.1 minutes. Overall, such a commit went from roughly 1.9 hours at the start of the semester to 3.1 minutes.

Contents

Certificate i

Acknowledgement ii

Abstract iii

1 Introduction 1
- 1.1 Research Questions . . . 1
- 1.2 Contributions . . . 1
- 1.3 Mapping to the Semester Plan . . . 1
- 1.4 Scope and Conventions . . . 2

2 System Architecture on AWS 2
- 2.1 End-to-End Pipeline . . . 2
- 2.2 Azure to AWS Service Mapping . . . 3
- 2.3 Baseline on AWS . . . 3

3 Datasets and Evaluation Setup 4
- 3.1 Datasets . . . 4
- 3.2 Machine and Software . . . 4
- 3.3 Measures . . . 5
- 3.4 Method . . . 5

4 Baseline Profiling 5
- 4.1 How Batching Worked . . . 5
- 4.2 Flaws Found . . . 6

5 Batching Optimization 6
- 5.1 Benchmark Method . . . 6
- 5.2 Batch Layout and Sorting . . . 6
- 5.3 Beam Size . . . 7
- 5.4 Parallel Batches and an Output Length Cap . . . 8
- 5.5 Adopted Configuration . . . 9

6 Placeholder Protection 9
- 6.1 The Problem . . . 9
- 6.2 Method . . . 9
- 6.3 Results . . . 10

7 Full KDE4 Production Run 10
- 7.1 Setup . . . 10
- 7.2 Results . . . 11
- 7.3 Comparison with the Starting Configuration . . . 12

8 Quality Problems Found at Scale 12
- 8.1 Characters the Model Cannot Write . . . 12
- 8.2 Options Measured . . . 13
- 8.3 Long Strings and the Translation Memory . . . 13

8.4 Verification . . . 13

9 Translation Memory Redesign **13**
9.1 Problems with the Previous Memory . . . 14
9.2 Design . . . 14
9.3 Quality Check of Near-Twin Reuse . . . 14

10 Translation Memory Experiment: Simulated CI/CD Commits **16**
10.1 Design . . . 16
10.2 Results per Step . . . 16
10.3 New Pipeline against the Previous Pipeline . . . 17
10.4 Quality, Lookups and Near-Twins . . . 18

11 Removing the Remaining Per-Run Costs **18**
11.1 Where a CI Run Spends Its Time . . . 18
11.2 Fixes . . . 19
11.3 Measured Before and After . . . 19
11.4 Contribution of Each Fix . . . 20

12 Semester Trajectory **20**

13 Engineering Decisions and Rationale **21**
13.1 Why beam 2 rather than beam 1? . . . 21
13.2 Why not larger batches or cross-file batching? . . . 21
13.3 Why square markers for placeholders? . . . 21
13.4 Why keep untranslatable strings in English? . . . 21
13.5 Why a hash as the memory key? . . . 21
13.6 Why record the engine version with each translation? . . . 21
13.7 Why base punctuation rules on counts? . . . 22
13.8 Why test the memory with commit-like versions? . . . 22
13.9 Why a stopped ready runner? . . . 22

14 Cost and Reliability **22**
14.1 Cost . . . 22
14.2 Reliability . . . 22

15 Limitations and Future Work **23**
15.1 Limitations . . . 23
15.2 Future Work . . . 23

16 Conclusion **23**

References **25**

A Telemetry Fields Added This Semester **25**

B Recorded Runs **25**

List of Figures

1 The localization pipeline on AWS . . . 3
2 Inference time per batching step . . . 7
3 Speed against quality by beam size . . . 8
4 Placeholders kept per language . . . 10
5 COMET per language in the full KDE4 run . . . 12
6 Near-twin reuse per language . . . 15
7 Time and model tokens per commit . . . 17
8 Composition of one 5% commit . . . 19
9 Semester trajectory of one 5% commit . . . 20

List of Tables

1 Semester plan mapping . . . 2
2 Azure to AWS service mapping . . . 3
3 First AWS run . . . 4
4 Datasets . . . 4
5 Measures . . . 5
6 Flaws of the original batching . . . 6
7 Batch layout and beam size . . . 6
8 chrF on FLORES-200 by beam size . . . 7
9 Beam size on the KDE4 sample . . . 7
10 chrF per language by beam size . . . 8
11 Parallel batches and output length cap . . . 8
12 Adopted batching configuration . . . 9
13 Placeholder survival by marker style . . . 10
14 Full KDE4 run per language: speed . . . 11
15 Translation quality of the full KDE4 run . . . 11
16 Handling of <unk> outputs . . . 13
17 Problems of the previous memory . . . 14
18 Near-twin reuse against fresh translation . . . 15
19 Application versions . . . 16
20 Every step of the memory experiment . . . 16
21 New pipeline against the previous pipeline . . . 17
22 Quality, lookups and near-twins . . . 18
23 Remaining per-run costs . . . 18
24 One 5% commit before and after the fixes . . . 19
25 Translation work on the same commits . . . 20
26 Sources of the trajectory times . . . 21
27 Machine cost per run . . . 22
28 New telemetry fields . . . 25
29 Recorded runs . . . 26

1 Introduction

Software products keep their user-facing text in resource files such as `.resx`. Releasing a product in several regions means translating every string into every supported language, and doing so again whenever the text changes. A localization pipeline automates this inside continuous integration: on every commit it extracts the strings, translates them with a neural machine translation (NMT) model, stores the results in a Translation Memory, and writes the localized resource files back.

The previous semester built such a pipeline on Microsoft Azure and established that, once the model runs locally, the pipeline is *compute-bound*: almost all of its time is spent inside the model. This semester asks the natural next question: *how much of that model work is necessary?* We approached it from two sides. First, make each model call more efficient (batching). Second, avoid model calls that repeat earlier work (Translation Memory). Along the way, a full-scale run on a real software-localization corpus exposed quality problems that only appear at scale, and those were fixed too.

1.1 Research Questions

RQ1. Where does the time go in the translation stage of the pipeline?

RQ2. Which batching changes improve token efficiency and speed, and what do they cost in translation quality?

RQ3. How does the optimized pipeline perform end to end on a twelve-language software-localization corpus?

RQ4. Which quality problems appear only at scale, and how can they be prevented?

RQ5. How much model work, time and cost does a persistent Translation Memory save under realistic CI/CD commits?

RQ6. Which set-up costs remain in every CI run, and how much does removing them save?

1.2 Contributions

- An AWS deployment of the pipeline, defined as infrastructure-as-code, with translation logic kept identical to the Azure version (§2).
- A profiling baseline and a one-change-at-a-time batching study covering batch layout, length sorting, beam size, parallel batches and an output length cap (§4, §5).
- Placeholder protection that keeps software placeholders such as `%1` and `{name}` intact (§6).
- A full production run on all 10,054 usable KDE4 strings in 12 languages (§7), and fixes for the quality problems it revealed (§8).
- A redesigned Translation Memory and a CI/CD-style experiment that quantifies its savings (§9, §10).
- Removal of the remaining per-run set-up costs, measured end to end (§11).

1.3 Mapping to the Semester Plan

Table 1 maps the Track 3 tasks of the semester plan to the chapters of this report.

Table 1: Track 3 (CI/CD & Memory Optimization) tasks and where they are addressed.

Phase	Task in the semester plan	Chapters
1 (weeks 1–2)	Profile pipeline bottlenecks; design new batching logic for token efficiency	4, 5
2 (weeks 3–4)	Implement optimized batching; build/update the Translation Memory to reduce lookups	5–9
3 (weeks 5–7)	Benchmark token-usage reduction, lookup speed-ups and cost savings	10–14

1.4 Scope and Conventions

- **Translation** means one string translated into one language; 100 strings in 12 languages are 1,200 translations.
- **Previous pipeline** means the pipeline before the change being discussed; each chapter states which version it compares against.
- **Measured** results come from recorded runs. **Estimated** results are calculated from measured rates and are always labelled as such.
- Translation quality is reported as **chrF** [4], **BLEU** and **spBLEU** against the reference translations, as **COMET** [6] on a fixed sample, and as the share of strings whose placeholders survive translation. The benchmark runs of Chapters 5 and 6 recorded chrF only.

2 System Architecture on AWS

2.1 End-to-End Pipeline

The pipeline keeps the three-phase shape of the previous semester (Figure 1):

1. **Phase 1 – Extract.** English `.resx` files are converted to XLIFF, one file per target language, and uploaded to Amazon S3.
2. **Phase 2 – Translate.** Each XLIFF file is checked against the Translation Memory in Amazon RDS (SQL Server). Strings not found are translated by NLLB-200 (distilled, 600M parameters) [1] running on the CTranslate2 engine [2] with INT8 quantisation inside a Docker container, and the new translations are saved back to the memory.
3. **Phase 3 – Merge.** The translated XLIFF files are merged back into localized `.resx` files and published to S3.

Every translated file emits one structured `RESEARCH_DATA` telemetry record to Amazon CloudWatch Logs, with the same schema as the previous semester plus the new fields listed in Appendix A.

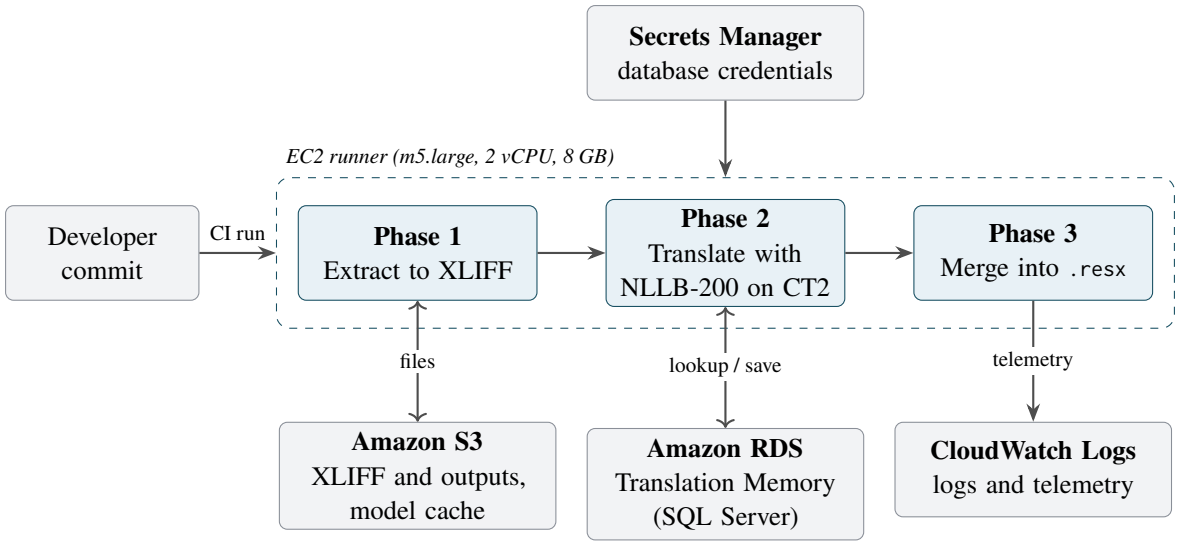


Figure 1: The localization pipeline on AWS. All resources are created by one CloudFormation stack in `ap-south-1`.

2.2 Azure to AWS Service Mapping

The move to AWS changed only the services around the translation code (Table 2). The translation logic itself was kept identical to the Azure version, and automated parity tests compare the two code bases so that any divergence is a deliberate, documented change.

Table 2: Azure services of the previous semester and their AWS replacements.

Role	Azure (previous semester)	AWS (this semester)
File storage	Blob Storage	Amazon S3 (`runs/<run-id>/raw`, `translated`, `output`)
Translation Memory	Azure SQL Database	Amazon RDS for SQL Server Express
Telemetry	Application Insights	Amazon CloudWatch Logs (same `RESEARCH_DATA` payload)
Credentials	pipeline variables	AWS Secrets Manager (RDS-managed password)
CI/CD orchestration	Azure DevOps Pipelines	EC2 runner driven by SSM Run Command[†]
Infrastructure	manual set-up	AWS CloudFormation (one stack)

[†]An AWS CodePipeline/CodeBuild definition is included, but CodeBuild is not enabled for this account, so runs use an EC2 runner with the same three phases.

2.3 Baseline on AWS

The first complete AWS run (`ec2-20261015t233954z`) translated the previous semester's 50-string workload into seven languages with an empty memory. All seven files completed with no failures (Table 3). This run is the starting point for the profiling in Chapter 4.

Table 3: First AWS run: 50 strings per language, empty Translation Memory, beam 5 (m5.large, 2 cores).

Language	File time (s)	Strings/hour	Model time (s)
de-DE	22.30	8,070	22.15
es-ES	21.19	8,492	21.04
fr-FR	22.73	7,920	22.59
hi-IN	21.71	8,291	21.57
ja-JP	20.41	8,820	20.27
ru-RU	22.10	8,143	21.97
zh-CN	19.47	9,247	19.32
Average	21.42	8,426	21.27

3 Datasets and Evaluation Setup

3.1 Datasets

Three workloads were used (Table 4). The main results use the KDE4 corpus [3], a collection of real user-interface strings from the KDE desktop software, prepared by the Track 1 team with reference translations in twelve languages.

Table 4: Datasets used in this report.

Dataset	Size	Used for
`.resx` workload	3 files, 153 strings, 7 languages (1,071 translations)	first batching benchmark
FLORES-200 dev [1]	first 40 sentences, 7 languages	quality check of beam sizes
KDE4	10,231 rows → **10,054 usable** strings, 12 languages	all main results

KDE4 preparation. The twelve target languages are Hindi (hi), Arabic (ar), Bengali (bn), German (de), Spanish (es), French (fr), Japanese (ja), Marathi (mr), Punjabi (pa), Tamil (ta), Telugu (te) and Chinese (zh). Of the 10,231 rows, 177 were removed: one cell containing hundreds of strings joined by line breaks, two cells that the spreadsheet had converted to Boolean values, and duplicated English strings. The usable strings are short (median 13 characters, about two words), typical of user-interface text. 355 of them contain placeholders such as `%1`, `%s`, `{name}` or `<b>`.

Reference translations. Several signs suggest that the KDE4 reference translations were produced by machine rather than by KDE's human translators: every row is filled in all twelve languages, "TRUE/FALSE" appears as "WAHR/FALSCH", and the references themselves drop `%N` placeholders in 5–25% of rows. chrF in this report therefore measures closeness to these references. Comparisons *between* pipeline versions remain fair, because all versions are scored against the same references; the placeholder measure does not use the references at all.

3.2 Machine and Software

All measurements ran on the same AWS instance type: **m5.large** (2 vCPU Intel Xeon Platinum 8259CL, 8 GB RAM, on-demand price $0.101 per hour in `ap-south-1`). The translation container is pinned to the machine's two cores. The model is `facebook/nllb-200-distilled-600M`, converted to CTranslate2 4.7.1

with INT8 weights.

3.3 Measures

Table 5: Measures reported in this report.

Measure	Meaning
Time, translations/hour	wall-clock time of the measured stage, and its throughput
Speed-up	time of the reference configuration divided by time of the new one
Model tokens	real tokens read and written by the model (tokenizer counts)
Padding	share of the model's input positions filled with padding inside batches
CPU busy	CPU time used divided by wall time × available cores
chrF	character n-gram F-score [4] against the reference translations, computed with SacreBLEU [5] (0–100, higher is better)
BLEU	word n-gram precision score against the references (SacreBLEU `13a`; Chinese `zh`; Japanese MeCab)
spBLEU	BLEU on FLORES-200 SentencePiece pieces [7], comparable across scripts
COMET	neural quality estimate (`Unbabel/wmt22-comet-da` [6], 0–1) on the same 1,000 random strings in every language
Placeholders kept	share of strings with placeholders whose output contains exactly the same placeholders
Cost	machine time × on-demand price

3.4 Method

Every optimization was evaluated **one change at a time**, starting from the best configuration found so far, so that each gain can be traced to its cause. Changes that should not alter the output (batch layout, parallelism, length caps) were checked for **identical translations**. Changes that do alter the output (beam size, placeholder handling, translation reuse) were scored for quality before being adopted.

4 Baseline Profiling

The telemetry of the first AWS run (Table 3) splits each file's time into its parts. On average a 50-string file took 21.42 s, of which **21.27 s (99.3%) was model inference**. Database lookups, S3 transfers and XML processing together took about 0.15 s. Any meaningful speed-up therefore has to come from the model stage: either making each model call cheaper, or making fewer of them.

4.1 How Batching Worked

Phase 2 processed one file and one language at a time. It filled a batch with untranslated strings in file order until the batch held 52 strings or an *estimated* 1,024 tokens (characters divided by four), sent it to the model with beam size 5 (five candidate translations per string), and waited for the result before sending the next batch.

4.2 Flaws Found

Table 6: Flaws of the original batching logic, and the evidence for each.

Flaw	Evidence
Guessed token counts	batches were cut by characters ÷ 4, not by the tokens the model actually reads
Mixed-length batches	short and long strings shared batches; **30%** of the model's input positions were padding
One batch at a time	only one batch was in the engine at any moment; the two cores were **59%** busy on average
Five candidates per string	beam 5 decodes five hypotheses for every string, the dominant cost (Chapter 5)
No output limit	a single string with an unusually long output could hold up its whole batch

RQ1. Model inference accounts for 99.3% of the translation stage. Within it, the original batching wasted input positions on padding (30%), left part of the CPU idle (59% busy) and paid for five candidate translations per string.

5 Batching Optimization

Batching decides how strings are grouped before they reach the model. This chapter tests each part of the batching logic separately: batch layout and sorting, beam size, and parallel execution with an output length cap.

5.1 Benchmark Method

A benchmark tool runs inside the production container, with the same model, the same two pinned cores and the same machine type, and times only model inference (tokenize, translate, detokenize). Its baseline step calls the production translation function itself, and a sanity check confirmed that the benchmark engine reproduces production output exactly (100 of 100 strings).

5.2 Batch Layout and Sorting

Table 7 applies the layout changes to the `.resx` workload (1,071 translations). A run-wide plan with real token counts and 1,024-token batches, sorted by length, cut padding from 30.1% to 4.3%. Smaller and larger token budgets were also tested.

Table 7: Batch layout and beam size on the `.resx` workload (run `ec2-20261016t103748z`). "Identical" compares the output with the baseline's output.

Step	Change	Beam	Time (s)	Speed-up	Padding (%)	Peak mem.	Identical
A	Original pipeline	5	434.84	1.00	30.1	1.8 GB	–
D-256	Sorted, 256-token batches	5	451.10	0.96	1.2	1.8 GB	100%
D-512	Sorted, 512-token batches	5	432.62	1.01	2.4	2.2 GB	100%
D-1024	Sorted, 1,024-token batches	5	425.78	1.02	4.3	3.5 GB	100%
F	D-1024 + beam 2	2	188.08	2.31	4.3	3.5 GB	77%
G	D-1024 + beam 1	1	103.58	4.20	4.3	3.5 GB	59%

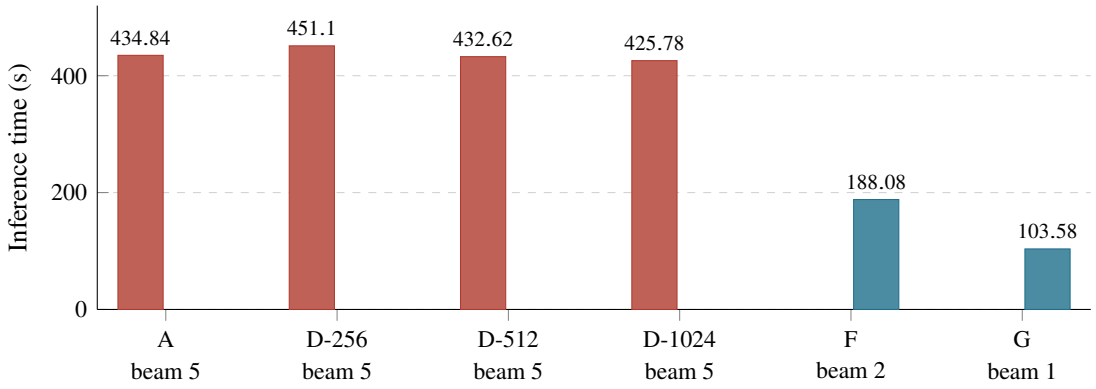


Figure 2: Inference time for each batching step. Layout changes (red) moved time by at most 4%; beam size (blue) halved it or more.

Finding. Layout changes never altered a translation, but moved time by only −4% to +2%, within the 2% run-to-run noise measured on identical work. Padding fell from 30% to 1–4% without a matching speed-up: **padding is not the bottleneck**. Peak memory, however, roughly doubled with 1,024-token batches, and a 4,096-token attempt did not complete on the 8 GB machine, so batches above about 1,024 tokens were ruled out. Telling CTranslate2 explicitly to use two threads per batch also changed nothing.

5.3 Beam Size

The beam size is the number of candidate translations the model keeps while decoding. Steps F and G show that it, not the layout, dominates the cost: beam 2 was 2.31× and beam 1 4.20× faster than the baseline. Because a smaller beam changes the output, quality was checked on two datasets.

FLORES-200. On 40 human-translated sentences per language (Table 8), beam 2 lost 0.87 chrF and beam 1 lost 1.40 chrF overall.

Table 8: chrF on FLORES-200 (first 40 dev sentences per language) by beam size.

Beam	All 7	de-DE	es-ES	fr-FR	hi-IN	ja-JP	ru-RU	zh-CN
5	56.69	62.71	55.54	68.75	57.65	33.83	58.35	28.58
2	55.82	61.93	55.63	67.05	56.98	34.77	56.71	26.32
1	55.29	60.08	55.73	67.76	55.03	33.39	57.33	25.07

KDE4. Forty sentences per language is a small sample, so the comparison was repeated on a fixed random sample of 1,000 KDE4 strings in all twelve languages (12,000 translations per beam), using run-wide, length-sorted 1,024-token batches (Tables 9 and 10).

Table 9: Beam size on 1,000 KDE4 strings × 12 languages (run `ec2-20261016t115802z`).

Beam	Time (s)	Translations/hour	Speed-up	chrF (all 12)	Identical to beam 5
5	3,169	13,631	1.00	64.41	–
2	1,442	29,949	2.20	63.96	84%
1	915	47,197	3.46	62.26	69%

Table 10: chrF per language on the KDE4 sample by beam size.

Beam	hi	ar	bn	de	es	fr	ja	mr	pa	ta	te	zh
5	66.90	57.15	64.87	65.06	72.01	70.23	53.43	59.34	63.35	64.70	66.07	39.68
2	66.61	55.74	64.11	64.39	71.69	69.48	52.52	59.08	63.27	65.07	65.86	39.30
1	65.60	53.77	63.88	62.21	68.49	68.16	46.51	58.76	62.68	63.70	64.26	33.72

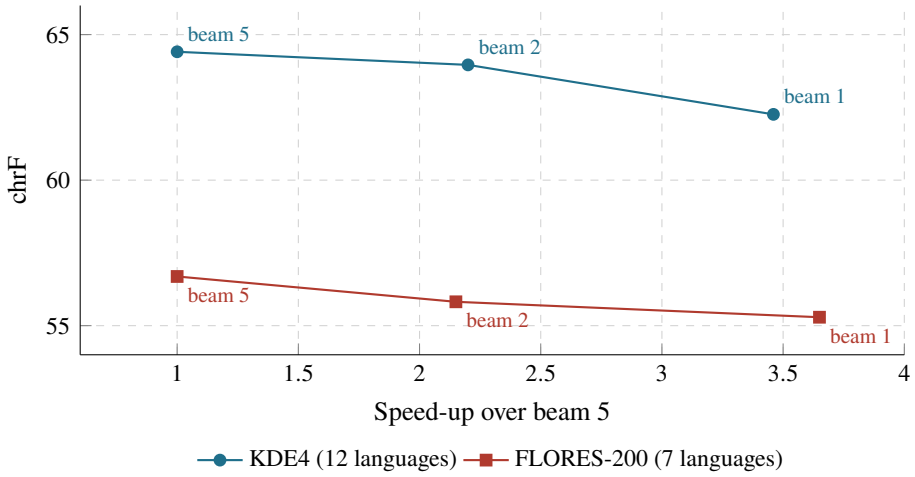


Figure 3: Speed against quality for beam sizes 5, 2 and 1. Beam 2 keeps almost all of the quality at more than twice the speed.

Finding. On real localization text, beam 2 is **2.2× faster for a 0.45-point chrF loss**, and every language stays within 1.4 points of beam 5 (Tamil slightly improves). Beam 1 costs more on KDE4 than on FLORES, with large drops for Japanese (−6.9) and Chinese (−6.0), which makes it unsuitable for these languages. Beam 2 was adopted for production.

5.4 Parallel Batches and an Output Length Cap

During the KDE4 runs the machine's CPU was only about 59% busy: CTranslate2 processed one batch at a time and part of the two cores sat idle. Three steps were tested on 500 KDE4 strings in 12 languages (6,000 translations, beam 2):

- **P1:** load the model with two engine workers (`inter_threads=2`, `intra_threads=1`) and queue all batches at once, so two batches are translated in parallel, one per core.
- **P2:** P1 with smaller 512-token batches.
- **P3:** P1 plus a per-batch output cap of twice the longest input plus 10 tokens. Because batches are sorted by length, the cap fits every string in the batch and only stops runaway outputs.

Table 11: Parallel batches and output length cap (run `ec2-20261016t142418z`, 6,000 translations, beam 2).

Step	Change	Time (s)	Speed-up	CPU busy (%)	Identical	Peak mem.
P0	One batch at a time	617.52	1.00	59.3	–	2.6 GB
P1	Two batches in parallel	514.97	1.20	99.6	100%	4.5 GB
P2	P1 with 512-token batches	539.68	1.14	99.3	99.9%	4.5 GB
P3	**P1 + output length cap**	488.14	1.27	99.6	**100%**	4.5 GB

Finding. Parallel batches raised CPU use from 59% to 99.6% and made translation 1.20× faster with identical output. The gain is below 2× because two cores sharing the work each run slower than one core alone, and it costs a second set of working buffers (2.6 to 4.5 GB, still safe on 8 GB). Smaller batches were slower, because more batches mean more per-batch overhead. The length cap added a further 6% (1.27× in total) without changing a single translation.

5.5 Adopted Configuration

Table 12: Batching configuration before and after this chapter.

Setting	Before	After
Order of strings	file order	sorted by length within each file, in chunks of 1,000 strings
Batch size	52 strings or ~1,024 *estimated* tokens	up to 1,024 *real* tokens
Execution	one batch at a time	all batches of a chunk queued, two engine workers
Output length	unlimited	2 × longest input + 10 tokens per batch
Beam size	5	2

RQ2. Padding and batch layout are not the bottleneck: cutting padding from 30% to 4% gained 3%. The real levers are the beam size (2.2× for −0.45 chrF on KDE4) and keeping both cores busy (1.27× with identical output). Together these are about 2.8× faster in the benchmark.

6 Placeholder Protection

Software strings contain placeholders that the program fills in at run time, such as `%1`, `%s`, `{name}` or `<b>`. A translation that drops or alters a placeholder is not merely a lower-quality translation: it produces a broken message in the released software.

6.1 The Problem

In the KDE4 beam study, about **12% of strings with placeholders** came back with a placeholder missing or changed, at every beam size (88.0% kept at beam 5, 87.7% at beam 2). The model treats `%1` as ordinary text and sometimes translates, moves or drops it. Because beam size barely affected the problem, it needed its own fix.

6.2 Method

Before translation, each placeholder is replaced by a numbered marker; after translation the markers are replaced by the original placeholders. The restore step accepts what the model does to a marker in practice (full-width brackets in Chinese and Japanese, spaces inside the marker, digits written in the target script) and reports whether every marker came back exactly once. If not, the string is translated again without markers and that result is used (the *fallback*). Three marker styles were compared on all 355 KDE4 placeholder strings in 12 languages (4,260 translations per style, beam 2):

`Could not find conduit %1.`	original
`Could not find conduit [0].`	square
`Could not find conduit §0§.`	section
`Could not find conduit {0}.`	curly

6.3 Results

Table 13: Placeholder survival by marker style (run `ec2-20261016t195057z`, 4,260 translations per style).

Style	Kept (%)	Kept with fallback (%)	Fallback used (%)	Time (s)
None (before)	87.70	–	–	649.3
Square `[0]`	99.15	99.88	0.85	716.1
Section `§0§`	80.66	97.91	19.34	749.5
Curly `{0}`	88.78	97.37	11.22	747.8

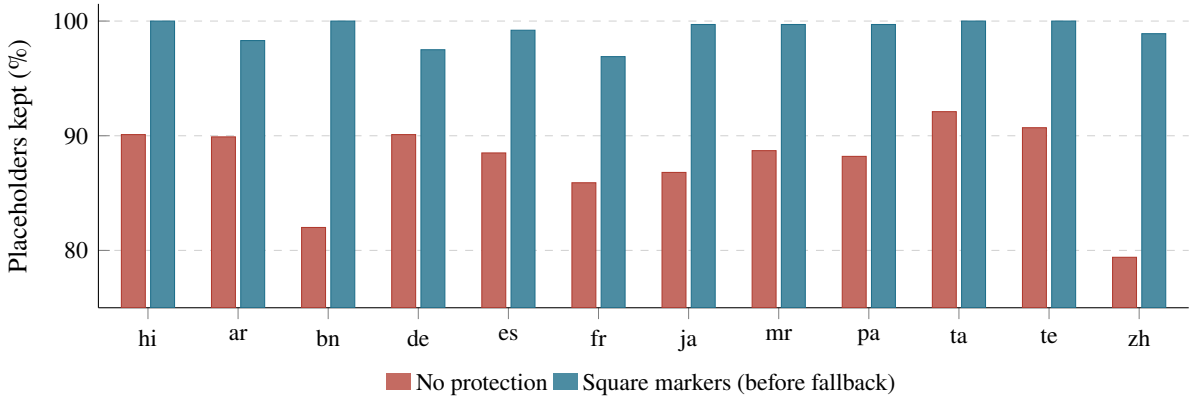


Figure 4: Placeholders kept per language, without protection and with square markers. Chinese improves the most (79.4% to 98.9%).

Finding. Square markers raise placeholder survival from **87.7% to 99.15%**, and to **99.88%** with the fallback: 5 broken strings out of 4,260 instead of about 524. Every language reaches at least 96.9% before the fallback. The square style works because NLLB saw many citation markers such as `[1]` during training and copies them unchanged; it often rewrites the other two styles. The cost is about 10% more time on strings with placeholders, which are 3.5% of KDE4, so the overall effect is negligible. Square markers with the fallback were adopted.

Note on chrF. On these 355 strings, chrF against the references fell by about two points (61.79 to 59.76 with fallback). The likely cause is that the references themselves drop `%N` placeholders in 5–25% of rows, so a translation that correctly keeps them matches those references less closely. Strings without placeholders are not affected by protection.

7 Full KDE4 Production Run

The benchmarks in Chapters 5 and 6 timed the model alone on samples of up to 1,000 strings. This chapter puts all the changes into the **production pipeline** (Phases 1–3, Translation Memory, S3, telemetry) and translates the **whole KDE4 dataset**.

7.1 Setup

All 10,054 usable KDE4 strings were packed into one `.resx` file and translated into all 12 languages: **120,648 translations**. The pipeline used the adopted batching configuration (Table 12), beam 2 and placeholder protection, on the same m5.large. The Translation Memory was emptied first, so every string went through the model (run `ec2-20261016t214343z`).

7.2 Results

The run succeeded: 12 of 12 language files, **0 failures, 0 missing strings**.

Table 14: Full KDE4 run per language: speed and placeholder survival, 10,054 strings per language.

Language	Time (min)	Translations/hour	Placeholders kept (%)
Arabic (ar)	18.28	32,991	99.72
Bengali (bn)	18.38	32,828	100.00
German (de)	17.83	33,841	99.44
Spanish (es)	17.62	34,232	100.00
French (fr)	19.00	31,742	99.72
Hindi (hi)	16.12	37,429	100.00
Japanese (ja)	16.18	37,282	100.00
Marathi (mr)	17.02	35,434	100.00
Punjabi (pa)	19.30	31,258	100.00
Tamil (ta)	18.77	32,144	100.00
Telugu (te)	19.15	31,502	100.00
Chinese (zh)	16.73	36,059	99.72
All	214.4	33,760	99.88

Table 15: Translation quality of the full KDE4 run. chrF, BLEU and spBLEU use all 10,054 strings per language; COMET uses the same fixed sample of 1,000 strings in every language.

Language	chrF	BLEU	spBLEU	COMET
Arabic (ar)	55.42	32.45	39.44	0.8296
Bengali (bn)	64.38	37.38	52.52	0.8866
German (de)	64.22	39.92	45.88	0.8639
Spanish (es)	71.50	50.65	53.15	0.8704
French (fr)	68.89	44.18	46.65	0.8495
Hindi (hi)	64.75	43.24	49.75	0.8526
Japanese (ja)	50.92	32.35	39.07	0.8643
Marathi (mr)	58.45	35.76	44.38	0.8249
Punjabi (pa)	61.53	38.56	50.42	0.8689
Tamil (ta)	64.10	37.75	49.53	0.8840
Telugu (te)	65.28	40.85	53.82	0.8932
Chinese (zh)	39.84	42.48	42.55	0.8530
All	63.29	**39.63***	47.90	0.8617

*Mean of the 12 languages: BLEU uses different word splitting for Chinese and Japanese, so a single corpus score across languages is not meaningful. spBLEU uses the same pieces for every language.

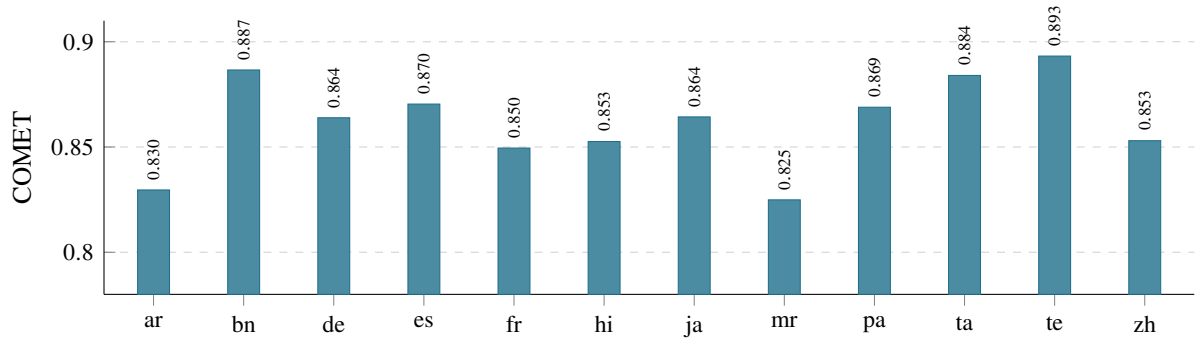


Figure 5: COMET per language in the full KDE4 run (1,000-string sample per language). Every language lies between 0.82 and 0.90.

Reading the metrics together. The four metrics agree on most languages, with one exception. chrF places Chinese far below every other language (39.84), but BLEU (42.48), spBLEU (42.55) and COMET (0.853) put it in the middle. chrF compares sequences of up to six characters, which is harsh for Chinese, where a word is often one or two characters; COMET, which judges meaning with a neural model, is the fairer comparison across scripts.

Where the time goes. Translation took **3 h 34 min** (3 h 38 min for the whole run, including model loading and Phases 1 and 3). Model inference accounted for 99.6% of Phase 2; Translation Memory reads and writes took about 44 s in total, and XML and S3 a few seconds. During translation the machine's CPU was 92–100% busy, and memory use stayed at 1.6–1.9 GB per file.

7.3 Comparison with the Starting Configuration

The starting configuration (beam 5, one batch at a time) ran at 13,631 translations per hour in the KDE4 benchmark (Table 9). At that rate this dataset would take an estimated **8.9 hours**; the optimized production pipeline needed **3.6 hours**, about **2.5× faster**. The benchmarks predicted 2.8×; the difference is the work the benchmark does not time: tokenization, placeholder handling, XML, database saves, and the end of each 1,000-string chunk, where only one batch is left for the two cores.

RQ3. The optimized pipeline translated the whole twelve-language KDE4 corpus (120,648 translations) in 3.6 hours with no failures, at 33,760 translations per hour, with chrF 63.29, spBLEU 47.90, COMET 0.862 and 99.88% of placeholders kept: about 2.5× faster than the semester's starting configuration.

8 Quality Problems Found at Scale

Translating the full corpus revealed two problems that the smaller tests could not show.

8.1 Characters the Model Cannot Write

Four strings in the full run came back *empty*: `thistle` (Japanese) and `Candle`, `crab` and `goy Y` (Chinese). Reproducing them with the same model showed the cause: for these strings the model's best output is the <unk> (unknown) token, and decoding removes special tokens, leaving nothing. The behaviour is the same at beam 5 and unrelated to batching.

Re-translating the whole Chinese and Japanese sets showed that the empty strings were only the visible part. **614 of 10,054 Chinese outputs (6.1%)** and 8 Japanese outputs contained <unk>. In most of them a character was silently deleted and the meaning changed: "New Folder" became "new file", "Dolphin Toolbar" became "fish tool", and "Bitrate" became just "rate".

8.2 Options Measured

Three ways of handling these strings were scored against the references on exactly the affected strings (Table 16).

Table 16: Handling of outputs that contain <unk> (chrF on the affected strings).

Option	zh (614)	ja (8)	Behaviour
Keep the model's text	26.20	6.61	characters silently missing; meaning often wrong
Forbid <unk> and retry	25.97	33.18	fixes some; others become "??" or a wrong word ("Tunnels" → "Tunisia", "thistle" → "carbohydrate")
Keep the English and report it	7.70	31.71	never a wrong word; listed in the log for a translator

Decision. The pipeline keeps such strings in English and reports them. In localized software, a visibly untranslated string can be found and fixed; a wrong word cannot. The cost is a lower measured chrF for Chinese, since chrF counts English as untranslated. Translation Memory entries for these strings are handled in Chapter 9.

8.3 Long Strings and the Translation Memory

The Translation Memory used the English text itself as its database key. SQL Server limits a key to 900 bytes, which allows at most 434 characters, while KDE4 contains strings of up to 660 characters; saving such a string failed. The key is now a fixed-size **SHA-256 hash** of the text, and the text is stored in an unlimited column, so strings of any length are saved and found again.

8.4 Verification

A check run (`ec2-20261017t063722z`) translated 383 KDE4 strings (the three longest, 25 known <unk> cases and all 355 placeholder strings) in 12 languages, twice, as two identical files. All 24 files succeeded. The second copy was served **100% from the memory** in ten languages, including the long strings; the only misses were the Chinese and Japanese strings kept in English, which were all listed in the log. Placeholder survival matched Chapter 6.

RQ4. At scale, 6.1% of Chinese outputs silently lost characters the model cannot write, and strings over 434 characters could not be stored in the memory. Affected strings are now kept in English and reported, and the memory is keyed by a hash so that strings of any length fit.

9 Translation Memory Redesign

A Translation Memory (TM) is a store of translations the pipeline has already made. Before sending a string to the model, the pipeline looks it up; a *hit* is reused instantly, and a *miss* is translated and saved for next time. The previous semester's memory worked, and measured a seeded 0% versus 50% cache. This chapter rebuilds it for the way the pipeline is actually used: on every commit of an evolving application.

9.1 Problems with the Previous Memory

Table 17: Problems of the previous Translation Memory.

Problem	Consequence
Emptied before every run	each commit retranslated the whole application, although a commit changes only a few strings
No check of which model made an entry	after a model or setting change, older translations would still be reused
One query per 1,000 strings per file	every file asked the database again for its own strings
Exact matches only	"8 Cards" could not reuse "20 Cards"; 526 KDE4 strings (5.2%) have such a near-twin

9.2 Design

Persistent, engine-labelled memory. The memory is kept between runs. Each entry records its *engine version*: the model and every setting that changes the output (beam size, length cap, placeholder style, handling of <unk>). Lookups match the current engine version, so a change of model or settings automatically stops older entries from being reused, without deleting anything. The table key is (language, engine version, text hash), with the language first so that one language's entries are stored together.

One load per language per run. At the first file of a language, the pipeline loads that language's entries for the current engine in a single query and keeps them in memory. Every later check is an in-process dictionary lookup, and translations made during the run are added as they are saved, so later files reuse them without another query.

Near-twin reuse. When the exact text is not found, the pipeline looks for a saved *model* translation of a string that differs only in numbers, placeholders or trailing punctuation, and adapts it:

Saved	New string	Adapted translation
`20 Cards` ⟶ `20 Karten`	`8 Cards`	`8 Karten`
`Copy %1 to %2` ⟶ `Nach %2 kopieren: %1`	`Copy %3 to %4`	`Nach %4 kopieren: %3`
`Border` ⟶ `Rahmen`	`Border:`	`Rahmen:`

Reuse is refused, and the string goes to the model as usual, whenever it cannot be done cleanly: a value does not appear exactly once in the saved translation (for example, the model wrote the number in local digits), the English repeats a value ("1 of 1"), or the saved entry was itself adapted. Twins inside one file are sent to the model once and the rest are adapted from that translation.

9.3 Quality Check of Near-Twin Reuse

Reuse only helps if adapted translations are as good as the model's own. Before enabling it, every twin pair in the full KDE4 set (440 groups, 968 strings, 528 reusable pairs) was translated in all twelve languages both ways: freshly by the production model and settings, and adapted from its twin. Both were scored against the references.

A first version *guessed* how each language writes trailing punctuation and scored lower than fresh translation for Japanese. Counting the endings of every KDE4 reference showed that only Chinese consistently uses the full-width colon, that Japanese uses the ASCII colon slightly more often (749 to 657), that "…" is

kept in every language, and that Hindi, Bengali and Punjabi end sentences with the danda while Marathi keeps the full stop. The punctuation rules now follow these counts. Table 18 reports the final rules.

Table 18: Near-twin reuse against fresh model translation (chrF on the reused strings, all 528 pairs per language).

Language	Reused	Refused	chrF fresh	chrF reused
Hindi	506	22	67.12	70.62
Arabic	488	40	55.51	56.08
Bengali	454	74	64.10	65.34
German	526	2	66.12	66.83
Spanish	527	1	66.82	68.04
French	510	18	67.54	68.74
Japanese	525	3	50.62	51.14
Marathi	492	36	57.62	59.55
Punjabi	527	1	60.24	62.60
Tamil	526	2	66.62	68.28
Telugu	526	2	66.98	70.50
Chinese	477	51	33.78	43.82
Mean of 12			60.26	62.63

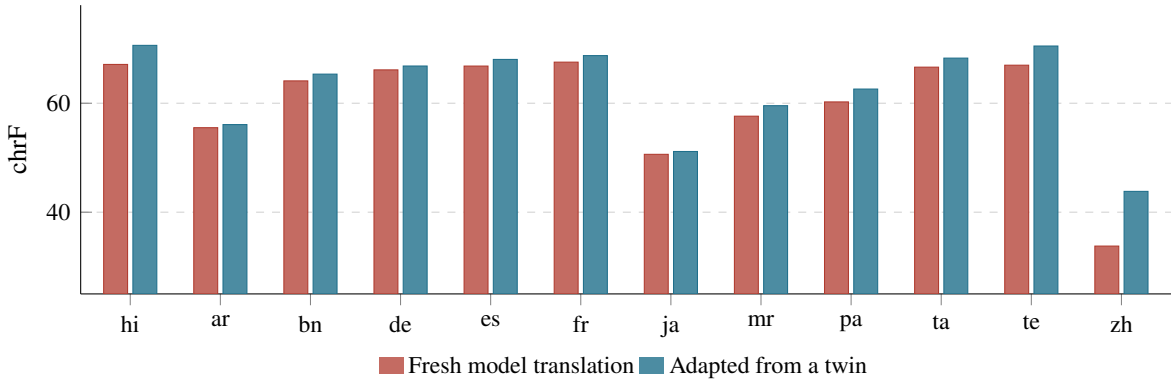


Figure 6: Near-twin reuse scores at least as well as fresh translation in every language.

Finding. By chrF, adapted translations score **equal or higher in all twelve languages** (mean 62.63 against 60.26), with placeholders kept 100% in both. chrF rewards consistency: the model translates each string in isolation and may translate "Points" one way in "42 Points" and another in "78 Points", while reuse applies the same translation to every twin. A COMET check on the strings where reuse actually changed the output (Chapter 10) shows **equal** quality rather than higher, so the benefit of reuse is saved model work and a more consistent application, at unchanged quality. Refusals (8%) were mostly strings whose numbers the model wrote in local digits; those strings were simply translated as before. Near-twin reuse is enabled by default.

10 Translation Memory Experiment: Simulated CI/CD Commits

10.1 Design

A Translation Memory pays off only *across* runs, when the same application is translated again after a change. The experiment therefore reproduces a commit history (Table 19): a first release, then commits that add 1%, 5% and 10% new strings. Each commit size was run three times (r1–r3), and every commit's new strings are used in no other version, so the memory has genuinely never seen them. The new strings are real KDE4 strings held back from the first release, so they have reference translations.

Table 19: Application versions used in the experiment (12 languages each).

Version	Content	Translations
v1	2,000 random KDE4 strings (first release)	24,000
v2	v1 + 20 new strings (1% commit)	24,240
v3	v1 + 100 new strings (5% commit)	25,200
v4	v1 + 200 new strings (10% commit)	26,400

All steps ran in sequence on one m5.large, keeping the memory between steps as a CI server would (run `ec2-20261017t175415z`). As the comparison point, the final step re-ran v4 with the memory emptied and near-twin reuse switched off, which is the pipeline as it was before this work. The previous pipeline's time for v2 and v3 was calculated from that step's measured rate (0.1046 s per translation). This is justified because, with nothing reused, time grows linearly with the number of translations: the first release, which also reused nothing, ran at 0.1039 s.

A 2,000-string application was used instead of all 10,054 strings to keep the experiment to about two and a half hours and to allow three repeats. The savings are proportions, so they do not depend on application size, and full-scale speed is known from Chapter 7.

10.2 Results per Step

Table 20: Every step of the experiment. "Work" is time spent translating the files; "Step" adds about 30 s of fixed start-up per run (image check, model download and load).

Step	Translations	Memory hits	Twins	To model	Work (s)	Step (s)	Model tokens
v1 (empty memory)	24,000	0	365	23,650	2,492.7	2,578.9	261,431
v2-r1 (+1%)	24,240	23,883	0	357	54.6	93.4	3,972
v3-r1 (+5%)	25,200	23,883	59	1,261	155.5	186.9	13,706
v4-r1 (+10%)	26,400	23,883	71	2,446	287.5	319.4	26,468
v2-r2	24,240	23,883	12	345	55.6	86.9	4,051
v3-r2	25,200	23,883	33	1,284	168.0	199.4	13,957
v4-r2	26,400	23,883	81	2,436	316.7	348.4	27,646
v2-r3	24,240	23,883	0	357	96.4	127.8	4,816
v3-r3	25,200	23,883	12	1,305	152.6	184.0	13,664
v4-r3	26,400	23,883	59	2,458	295.0	327.3	27,746
v4, previous pipeline	26,400	0	0	26,415	2,761.1	2,794.2	290,622

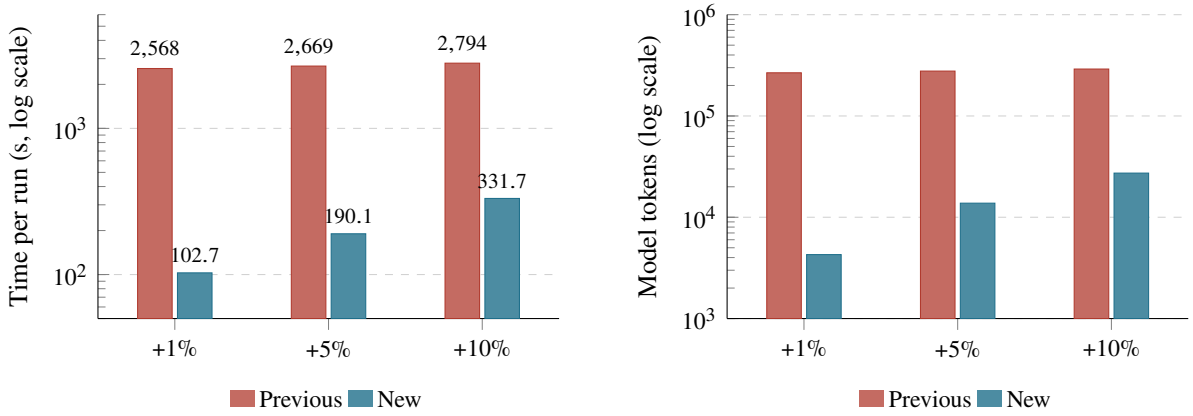
All 132 language files succeeded. The 117 misses among the unchanged strings in each later step are the Chinese and Japanese strings kept in English (Chapter 8), which were not yet remembered at this stage

(Chapter 11).

10.3 New Pipeline against the Previous Pipeline

Table 21: Averages of three repeats against the previous pipeline. Previous-pipeline values for +1% and +5% are calculated from its measured rate; +10% is measured. Cost uses $0.101 per machine-hour.

Commit	Previous (s)	New (s)	Speed-up	Tokens, prev.	Tokens, new	Saved (%)	Cost per run
+1%	2,568	102.7	25.0	266,844	4,280	98.4	$0.072 → $0.0029
+5%	2,669	190.1	14.0	277,412	13,776	95.0	$0.075 → $0.0053
+10%	2,794	331.7	8.4	290,622	27,287	90.6	$0.078 → $0.0093



(a) Time per run

(b) Model tokens per run

Figure 7: The previous pipeline's cost is fixed by application size; the new pipeline's cost follows the size of the change.

Why smaller commits gain more. The previous pipeline retranslated everything, so its time was about 45 minutes whatever changed. The new pipeline's time follows the size of the change, so the ratio is largest for small commits, which are the common case in CI/CD. A 1% commit did not reach the ideal 100× for three reasons, each addressed in Chapter 11: a fixed start-up of about 30 s per run; the 117 kept-English strings, which were sent to the model again on every run (a third of a 1% commit's model work); and small batches, which used the model less efficiently (58–85% padding against 16% when translating everything).

10.4 Quality, Lookups and Near-Twins

Table 22: Quality, database lookups and near-twin reuse.

Measure	Result
Quality, v4 from the memory	chrF **63.38**, spBLEU 48.07, BLEU 40.52, placeholders kept 100%, 0 missing
Quality, v4 previous pipeline	chrF **63.37**, spBLEU 48.04, BLEU 40.45, placeholders kept 100%, 0 missing
Strings that differ (170 of 26,400)	COMET 0.8654 from the memory against 0.8689 from scratch; each version better on 80 and 81 strings
Database queries per run	**12** (one per language) against 36 for the previous per-file, per-1,000-string queries
Time to load the memory	224–272 ms in total for all 12 languages; every later check is in-process
Near-twins in the first release	365 of 24,000 translations (1.5%) reused instead of translated

Quality is unchanged because a memory hit returns exactly the translation that the same engine produced earlier, which the engine label guarantees. The two v4 runs differ in only 170 of 26,400 translations, the strings adapted from near-twins, and on exactly those strings COMET is equal within noise (0.8654 against 0.8689, an 80–81 split). Near-twins were rarer in the 2,000-string application (1.5%) than in the full corpus (5.2%), because a twin helps only when its partner is also present.

RQ5. Under simulated CI/CD commits, the redesigned memory made 1%, 5% and 10% commits 25×, 14× and 8.4× faster than the previous pipeline, with 98.4%, 95.0% and 90.6% fewer model tokens and unchanged quality (chrF 63.38 against 63.37, spBLEU 48.07 against 48.04). Database queries fell from 36 to 12 per run.

11 Removing the Remaining Per-Run Costs

11.1 Where a CI Run Spends Its Time

The experiment in Chapter 10 ran on one machine that stayed on between steps. That measures the memory fairly, but it hides the cost a real CI run pays on *every* commit: starting a machine and preparing it. Timing a small commit end to end showed four remaining costs (Table 23).

Table 23: Per-run costs that remained after the Translation Memory redesign.

Part of one CI run	Time	Cause
Start and stop a fresh machine	~106 s	a new machine boots and installs Docker for every commit
Image build and model set-up	~80 s	the container image is built from scratch; the 629 MB model was copied into the image although it is mounted at run time; the model was downloaded and test-loaded every run
Kept-English strings	~23 s	strings stored as untranslatable were not remembered, so 117 went back to the model on every run with the same result
Small model calls	–	~30 new strings per language form one batch, so one of the two engine workers is idle

11.2 Fixes

1. **Model kept out of the image build.** A build-ignore rule excludes the model folder, which the translator mounts at run time.

2. **Kept-English strings remembered.** They are saved with an empty translation for the current engine version. Later runs skip the model for them and still list them for a translator; a new engine version retries them.

3. **Ready runner.** Instead of a fresh machine per commit, one runner is kept *stopped* between runs and started when needed. Its disk keeps the built image and the model, so a run skips the build and the download, and a model already present for the current version also skips its test load. The runner is stopped, not deleted, after each run.

4. **Balanced small batches.** A model call small enough for two batches is split into two length-sorted batches of equal tokens, one per engine worker. Large calls are unchanged.

11.3 Measured Before and After

One 5% commit was timed end to end, from launch to result, with the code before these fixes on a fresh machine (as CI ran until then) and with the current code on the ready runner, on the same day and machine type (runs `ec2-20261018t143022z` and `ec2-20261018t143618z`).

Table 24: One 5% commit end to end, before and after the fixes. Both runs completed all 12 files with no failures.

Part	Before (s)	After (s)	Saved (s)
Start and stop the machine	106.1	37.5	68.6
Image and model set-up	79.5	18.7	60.8
Translation work	168.1	127.2	40.9
Phases 1 and 3	2.9	2.8	0.1
Whole run	356.6	186.2	170.4
Strings sent to the model	1,284	1,188	
Machine cost of the run	$0.0100	$0.0052	

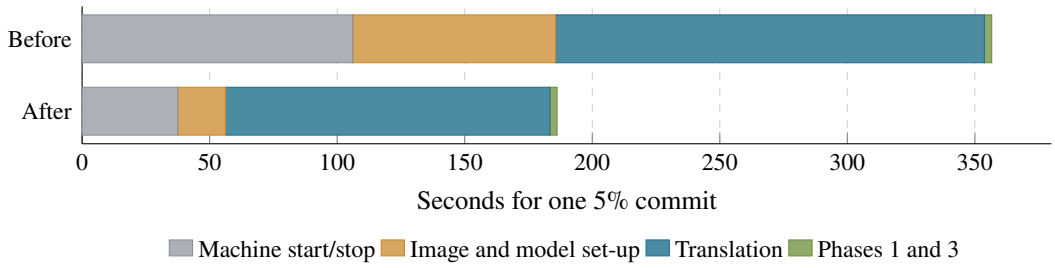


Figure 8: Composition of one 5% commit before and after the fixes: 5.9 minutes to 3.1 minutes (1.9×).

Each commit must contain strings that are new to the memory, so the two runs used different 5% versions (v3-r2 and v3-r3); with the previous code, v3-r3 needed about 9% less translation work than v3-r2. Table 25 therefore compares translation work on the *same* commits.

Table 25: Translation work on the same commits with the previous code (Chapter 10) and the current code.

Commit	Strings to model	Before (s)	After (s)	Change (%)
+1% (v2-r1)	357 → 240	54.6	38.2	−30
+1% (v2-r2)	345 → 228	55.6	39.2	−29
+1% (v2-r3)	357 → 240	96.4	74.5	−23
+5% (v3-r1)	1,261 → 1,144	155.5	127.4	−18
+5% (v3-r3)	1,305 → 1,188	152.6	127.2	−17
+10% (v4-r1)	2,446 → 2,329	287.5	252.1	−12

11.4 Contribution of Each Fix

- **Ready runner and image fix:** machine start and stop fell from about 106 s to 37–44 s, and image and model set-up from about 80 s to 19–36 s (four runs on the ready runner). Together this saves about two minutes on every CI run, whatever the size of the commit: the largest gain.
- **Remembered kept-English strings:** exactly 117 fewer strings sent to the model per commit; all 117 were found in the memory instead. This is most of the saving on small commits.
- **Balanced small batches:** about 8–10% less time per translated string on 5% and 10% commits (0.123 to 0.111 s, and 0.118 to 0.108 s), but no measurable gain on 1% commits, where fixed per-file costs outweigh model time.

With all fixes, measured end to end on the ready runner, a 1% commit took 156.9 s, a 5% commit 186.0–186.2 s and a 10% commit 312.2 s. The stopped runner's 40 GB disk costs $3.65 per month whether it is used or not.

RQ6. Starting a fresh machine, rebuilding the image and re-sending untranslatable strings cost about three minutes per CI run. Removing them cut a 5% commit, measured end to end, from 5.9 to 3.1 minutes (1.9×).

12 Semester Trajectory

Figure 9 follows one 5% commit (2,100 strings, 25,200 translations) through the semester. Each stage changes one part of the pipeline.

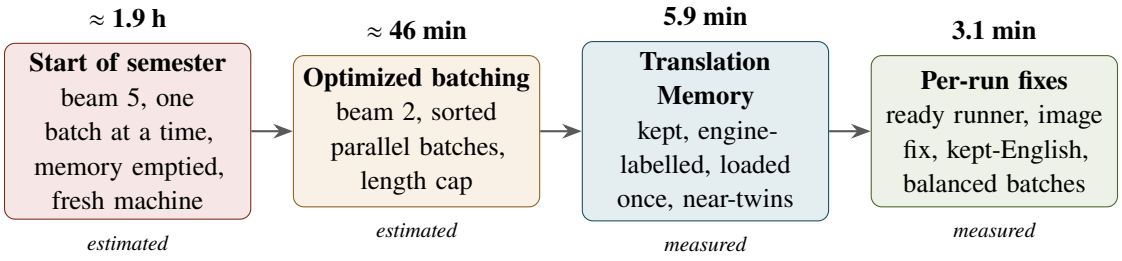


Figure 9: One 5% commit, end to end, across the semester's stages.

Table 26: How each stage's time for a 5% commit was obtained.

Stage	Time	Source
Start of semester	≈1.9 h	25,200 translations at the benchmark rate of 13,631 per hour (Table 9), plus machine start and set-up
Optimized batching	≈46 min	25,200 translations at the previous pipeline's measured 0.1046 s each (Chapter 10), plus machine start
Translation Memory	5.9 min	measured end to end on a fresh machine (Table 24, "before")
Per-run fixes	3.1 min	measured end to end on the ready runner (Table 24, "after")

Batching made every model call cheaper (about 2.5× at full scale); the Translation Memory removed most model calls altogether; and the per-run fixes removed most of the set-up around them. Overall, a typical commit went from roughly 1.9 hours to 3.1 minutes, while quality was preserved at every step: identical output for batching layout and parallelism, a 0.45-point chrF trade-off for beam 2, more placeholders kept, and unchanged chrF, BLEU and COMET with the memory.

13 Engineering Decisions and Rationale

13.1 Why beam 2 rather than beam 1?

Beam 1 is faster (3.46× against 2.20× on KDE4) but loses 2.15 chrF overall and about six points for Chinese and Japanese. Beam 2 keeps every language within 1.4 points of beam 5 for more than twice the speed.

13.2 Why not larger batches or cross-file batching?

Batch layout changed speed by at most 3% while doubling peak memory, and a 4,096-token attempt did not complete on the 8 GB machine. Batches are therefore kept at 1,024 tokens and per file, which also preserves per-file telemetry.

13.3 Why square markers for placeholders?

They survived translation in 99.15% of cases against 88.78% and 80.66% for the alternatives, because the model copies citation-style markers such as `[1]` unchanged.

13.4 Why keep untranslatable strings in English?

Forcing the model to avoid `<unk>` produced fluent but wrong words. An English string in a localized application is visible and can be fixed by a translator; a wrong word is silent.

13.5 Why a hash as the memory key?

SQL Server limits a key to 900 bytes (434 characters). A fixed 32-byte SHA-256 hash removes the limit while keeping lookups indexed.

13.6 Why record the engine version with each translation?

It makes reuse safe across model and setting changes without manual cleanup: older entries simply stop matching.

13.7 Why base punctuation rules on counts?

The first, guessed rules made Japanese reuse worse than fresh translation. Counting how every reference ends brought every language to equal or better chrF than fresh translation, and COMET confirmed equal quality.

13.8 Why test the memory with commit-like versions?

A memory only pays off across runs. Versions with genuinely new strings, run in sequence with the memory kept, measure the savings a CI server would actually see, rather than a seeded cache rate.

13.9 Why a stopped ready runner?

A stopped machine keeps its disk (the built image and the model) for $3.65 per month and starts in seconds, saving about two minutes per CI run over a fresh machine.

14 Cost and Reliability

14.1 Cost

All costs below are machine time at the on-demand price of an m5.large in `ap-south-1` ($0.101 per hour). The database and storage cost the same regardless of the pipeline version and are not included.

Table 27: Machine cost per run.

Run	Machine time	Cost
Full KDE4 corpus, 12 languages (120,648 translations)	3 h 38 min	$0.367
per 1,000 translations		$0.0030
5% commit, previous pipeline (memory emptied, calculated)	≈46 min	≈$0.075
5% commit, new memory, fresh machine (measured)	5.9 min	$0.0100
5% commit, all fixes, ready runner (measured)	3.1 min	$0.0052
Ready runner disk while stopped (40 GB gp3)	–	$3.65 per month

Per run the amounts are small, but a CI pipeline runs on every commit, often many times a day and for many applications and languages, so a 93% saving per commit compounds. The larger benefit is time: developers receive localized resources in about three minutes instead of most of an hour.

14.2 Reliability

Every recorded run in this report completed without translation failures: 12 of 12 files in the full KDE4 run, 24 of 24 in the check run, 132 of 132 in the memory experiment, and 12 of 12 in each end-to-end commit. Phase 2 also treats any swallowed inference error, missing segments or storage error as a failed file instead of a silent success, isolates failures per file so that one bad file does not stop the others, and keeps unprocessed files in S3 for a retry. The code is covered by an automated test suite of 254 tests.

15 Limitations and Future Work

15.1 Limitations

- **Reference translations.** The KDE4 references appear to be machine-generated (Chapter 3), so chrF measures closeness to them rather than to human translation. Comparisons between pipeline versions are unaffected because all versions use the same references.
- **Calculated baselines.** The previous pipeline's times for 1% and 5% commits, and the start-of-semester stage in Chapter 12, are calculated from measured rates; each is labelled.
- **One machine type and short strings.** All timings are for an m5.large and for user-interface strings with a median length of two words. Longer text would change absolute times, not the conclusions about what dominates them.
- **Experiment size.** The memory experiment used a 2,000-string application, in which near-twins are rarer than in the full corpus (1.5% against 5.2%).
- **Metric coverage.** COMET was computed on a fixed 1,000-string sample per language (12,000 translations) rather than all 120,648, because CPU scoring of the full set takes several hours. The benchmark runs recorded chrF only. ROUGE-L and METEOR are computed by Track 1.

15.2 Future Work

- Repeat the memory experiment at full scale (10,054 strings) to confirm the savings on a production-size application.
- Compare machine types, including newer CPUs and GPUs, by cost per 1,000 translations.
- Evaluate beam 1 for the languages where its loss was smallest (under 1.3 chrF for Hindi, Marathi, Punjabi and Bengali), keeping beam 2 for Chinese and Japanese.
- Store the memory as per-language files in S3 now that it is loaded once per run, removing the always-on database.

16 Conclusion

In the first half of this semester, the localization pipeline was moved to AWS and the work it does on every commit was reduced. Profiling showed that model inference takes 99.3% of the translation stage. A one-change-at-a-time study then showed that the common intuition about batching, that less padding means more speed, does not hold here: padding fell from 30% to 4% for a 3% gain, while the beam size and idle cores were the real costs. With beam 2, parallel batches and an output length cap, the production pipeline translated the full twelve-language KDE4 corpus, 120,648 translations, in 3.6 hours with no failures, about 2.5× faster than at the start of the semester.

Scale exposed problems that small tests could not: broken placeholders and characters the model cannot write. Placeholder protection raised survival from 87.7% to 99.88%, and untranslatable strings are now kept in English and reported rather than shipped broken.

The redesigned Translation Memory changed the economics of the pipeline. Because it persists, records the engine that produced each entry, loads once per language and reuses near-identical strings, the cost of a run now follows the size of the change rather than the size of the application: 1%, 5% and 10% commits ran 25×, 14× and 8.4× faster with 90–98% fewer model tokens and unchanged quality by chrF, BLEU

and COMET. Removing the remaining per-run set-up brought a typical 5% commit to 3.1 minutes end to end, from roughly 1.9 hours at the start of the semester.

References

[1] NLLB Team, M. R. Costa-jussà, J. Cross, et al. No Language Left Behind: Scaling Human-Centered Machine Translation. *arXiv:2207.04672*, 2022.

[2] OpenNMT. CTranslate2: Fast inference engine for Transformer models. https://github.com/OpenNMT/CTranslate2.

[3] J. Tiedemann. Parallel Data, Tools and Interfaces in OPUS. In *Proceedings of LREC*, 2012.

[4] M. Popović. chrF: character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, 2015.

[5] M. Post. A Call for Clarity in Reporting BLEU Scores. In *Proceedings of the Third Conference on Machine Translation*, 2018.

[6] R. Rei, J. G. C. de Souza, D. Alves, et al. COMET-22: Unbabel-IST 2022 Submission for the Metrics Shared Task. In *Proceedings of the Seventh Conference on Machine Translation*, 2022.

[7] N. Goyal, C. Gao, V. Chaudhary, et al. The FLORES-101 Evaluation Benchmark for Low-Resource and Multilingual Machine Translation. *Transactions of the Association for Computational Linguistics*, 10, 2022.

A Telemetry Fields Added This Semester

Each translated file emits one `RESEARCH_DATA` record. The previous semester's fields are unchanged; Table 28 lists the fields added this semester.

Table 28: New `RESEARCH_DATA` fields.

Field	Meaning
`cloud_provider`, `region`, `run_id`	where and in which run the file was translated
`cpu_limit`, `cpu_model`, `host_cpu_count`	cores available to the translator and the host CPU
`tm_db_queries`	database queries made for this file (0 after the language is loaded)
`tm_lookup_ms`	time spent on memory lookups
`tm_hits_in_run`	hits on entries translated earlier in the same run
`tm_twin_hits`	strings adapted from a near-twin
`tm_kept_english`	strings remembered as kept in English
`model_strings`	strings sent to the model, including placeholder fallbacks
`real_input_tokens`, `real_output_tokens`	tokens read and written by the model (tokenizer counts)
`padding_pct`	share of padded input positions in this file's batches

B Recorded Runs

All runs used an m5.large in `ap-south-1`. Outputs are stored in S3 under `runs/<run-id>/`, and logs and telemetry in CloudWatch Logs.

Table 29: Runs referenced in this report.

Run	Content	Chapter
`ec2-20261015t233954z`	first AWS run, 50 strings × 7 languages	2
`ec2-20261016t103748z`	batch layout and beam size, `.resx` workload and FLORES-200	5
`ec2-20261016t115802z`	beam size on 1,000 KDE4 strings × 12 languages	5
`ec2-20261016t142418z`	parallel batches and output length cap	5
`ec2-20261016t195057z`	placeholder marker styles	6
`ec2-20261016t214343z`	full KDE4 production run	7
`ec2-20261017t063722z`	check run: long strings, `<unk>`, memory hits	8
`ec2-20261017t175415z`	Translation Memory experiment (11 steps)	10
`ec2-20261018t132556z` – `t142626z`	commits with the per-run fixes	11
`ec2-20261018t143022z`, `t143618z`	5% commit before and after the fixes	11